Tema 2.1: Desarrollo de aplicaciones para móviles – Estructura y controles

Laura Sacristán Matesanz / David López Cuenca

Objetivos del tema

- Entender la estructura de una aplicación móvil
- Profundizar en el Desarrollo de aplicaciones móviles
- Conocer controles avanzados de las aplicaciones móviles

Debug de una aplicación Android I

- Nos ayuda a depurar un programa cuando se produce algún fallo o error y detectar dónde se está produciendo.
- Para ello utilizaremos los BREAK POINT.
- Break Point: es un punto del programa donde queremos que se detenga el programa para poder inspeccionar los valores de las variables.

Debug de una aplicación Android II

- Vamos a utilizar el ejemplo del ciclo de vida para ver como funcionan los BreakPoint.
- Lo primero que vamos a hacer es añadir un variable privada de tipo integer y la incializaremos a 0 en el método onCreate.

```
HelloActivity onCreate()
3   import android.support.v7.app.AppCompatActivity;
4   import android.os.Bundle;
5   import android.util.Log;
6
7   public class HelloActivity extends AppCompatActivity {
8       private int i;
9
10      @Override
11      protected void onCreate(Bundle savedInstanceState) {
12          super.onCreate(savedInstanceState);
13          setContentView(R.layout.activity_hello);
14
15          Log.i("TAG cicloVida ","Ciclovida: onCreate");
16
17          i = 0;
18      }
```

activity_main.xml | MainActivity.java

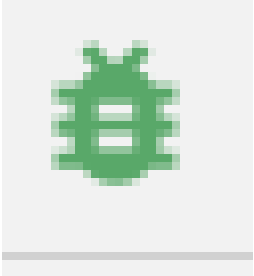
```java
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        i = 0;
    }
    @Override
    protected void onStart (){
        super.onStart();
        Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onStart");
        i++;
    }
    @Override
    protected void onResume (){
        super.onResume();
        Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onResume");
        i++;
    }
    @Override
    protected void onPause (){
        super.onPause();
        Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onPause");
        i++;
    }
    @Override
    protected void onStop (){
        super.onStop();
        Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onStop");
        i--;
    }
}
```

Debug de una aplicación Android III

- A continuación, en cada uno de los métodos que creamos en su momento, onStart, onResume, onPause, onStop y onDestroy vamos a ir incrementando la variable i para ver el valor que tiene.
- En el caso de los método onStop y onDestroy la decrementaremos para ver la diferencia.

Debug de una aplicación Android IV

- Una vez modificado el código, vamos a ejecutar la aplicación en modo Debug. Para ello pulsaremos el siguiente icono, que aparece en la barra de herramientas de la aplicación.



- Antes de pulsar esta opción tendremos que poner distintos puntos de BreakPoint para que si el programa pasa por ese punto se pare y podamos comprobar el valor de las variables de nuestro programa.

Debug de una aplicación Android V

- Vamos a poner un BreakPoint en cada uno de los métodos, por ejemplo donde tenemos el log. Para poner esos puntos tendremos que pulsar en el margen izquierdo donde queramos ponerlo.

activity_main.xml | MainActivity.java

```
10
11      @Override
12      protected void onCreate(Bundle savedInstanceState) {
13          super.onCreate(savedInstanceState);
14          setContentView(R.layout.activity_main);
15          i = 0;
16      }
17      @Override
18      protected void onStart (){
19          super.onStart();
20          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onStart");
21          i++;
22      }
23      @Override
24      protected void onResume (){
25          super.onResume();
26          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onResume");
27          i++;
28      }
29      @Override
30      protected void onPause (){
31          super.onPause();
32          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onPause");
33          i++;
34      }
35      @Override
36      protected void onStop (){
37          super.onStop();
38          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onStop");
39          i--;
40      }
41  }
```

Debug de una aplicación Android VI

- Pulsamos en la opción de debug y nos pedirá el emulador en el que queremos probarlo. Cada uno arrancará el que tenga instalado.
- Nos aparecerá la siguiente información:
  - El programa para en el primer BreakPoint marcado.
  - Aparece el valor que tiene nuestra variable i. En este punto la variable i solo está definida y al ser un integer su valor es cero.
  - También aparece el objeto This en el que se incluyen todas las propiedades definidas a nivel de clase. Aquí también aparece nuestra variable i.

File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help Aplicacion Hello Word - MainActivity.java [Aplicacion_Hello_Word.app]

AplicacionHelloWord > app > src > main > java > com > example > aplicacionhelloword > MainActivity

app | Nexus 6 API 30

Android

- app
  - manifests
  - java
    - com.example.aplicacionhelloword
      - MainActivity
    - com.example.aplicacionhelloword (androidTest)
    - com.example.aplicacionhelloword (test)
  - java (generated)
  - res
  - res (generated)
- Gradle Scripts

activity_main.xml | MainActivity.java

```
10
11      @Override
12      protected void onCreate(Bundle savedInstanceState) {   savedInstanceState: null
13          super.onCreate(savedInstanceState);   savedInstanceState: null
14          setContentView(R.layout.activity_main);
15          i = 0;
16      }
17      @Override
18      protected void onStart (){
19          super.onStart();
20          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onStart");
21          i++;
22      }
23      @Override
24      protected void onResume (){
25          super.onResume();
26          Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onResume");
27          i++;
28      }
29      @Override
30      protected void onPause (){
31          super.onPause();
```

Debug: app

Debugger | Console

Frames

"main"@15,7...n": RUNNING

- onCreate:14, MainActivity (com.example.aplicacionhe
- performCreate:7994, Activity (android.app)
- performCreate:7978, Activity (android.app)
- callActivityOnCreate:1309, Instrumentation (android.
- performLaunchActivity:3422, ActivityThread (android
- handleLaunchActivity:3601, ActivityThread (android.

Variables

- this = {MainActivity@16078}
- savedInstanceState = null
- i = 0

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Run | Debug | Event Log | Layout Inspector

Launch succeeded (12 minutes ago) 14:1 LF UTF-8 4 spaces

Debug de una aplicación Android VII

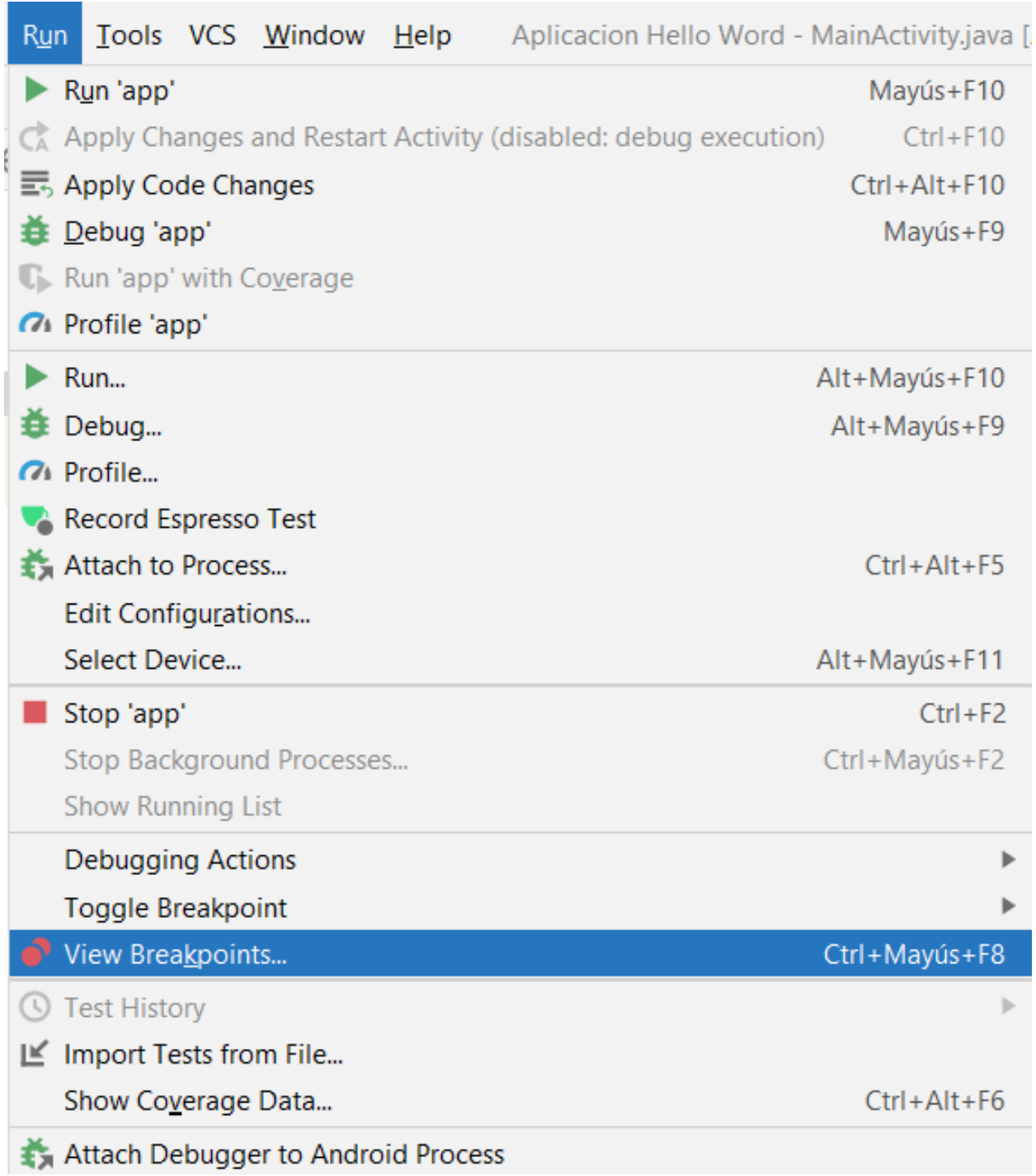
- Dentro de la opción Debug hay varias opciones:

- Siguiente línea de código
- Seguir un paso hacia adentro en el caso de que queramos entrar en el método
- Volver al BreakPoint en el que estamos
- Siguiente punto de interrupción

- Como ya vimos en el ciclo de vida de una aplicación, podemos ir viendo los resultados en la opción logcat y elegir ver los TAG.

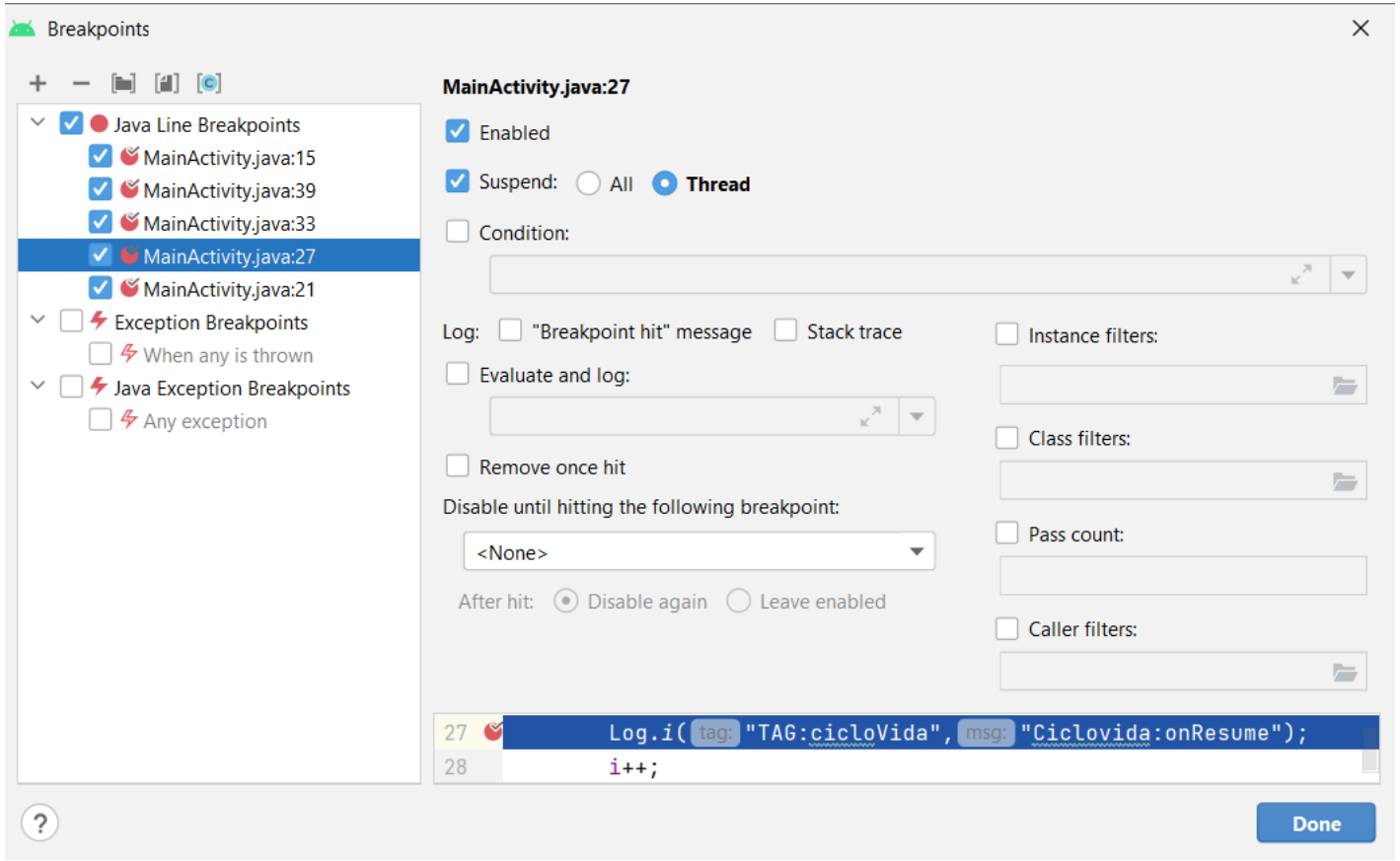
Debug de una aplicación Android VIII

- En la opción de Debug en la zona de variables iremos viendo el resultado de las variables.
- Si tenemos muchos puntos de interrupción, podemos gestionarlos a través de la opción run del menú y elegir la opción View Breakpoint



Debug de una aplicación Android IX

- Cuando pulsamos esa opción nos aparece la siguiente pantalla, desde la que podemos gestionar esos puntos de interrupción.



Creación de un Activity Secundario

Activity secundario I

- Para crear una nueva pantalla en nuestra aplicación para móviles necesitaremos tener un Activity Secundario.
- Necesitaremos saber también como pasar de una pantalla a otra para poder utilizarlo en nuestra aplicación.
- Cuando nosotros abrimos una aplicación nueva desde cero, únicamente tenemos un Activity Principal que viene indicado en nuestro AndroidManifest con la etiqueta <activity> y dentro de ella la etiqueta <intent-filter>

activity_main.xml | MainActivity.java | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.aplicacionhelloword">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="Aplicacion Hello Word"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AplicacionHelloWord">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Activity secundario II

- Para que los Activity existan deben estar declarados en el archivo AndroidManifest.
- Todos los ficheros .java que definen un activity deben estar declarados en el archivo AndroidManifest. Si no están declarados es como si no existieran.
- Para crear un nuevo Activity podemos hacerlo desde la carpeta app, botón derecho y elegir la opción new y dentro de new Activity.
- Elegiremos la opción Empty Activity, que es el modelo que hemos seleccionado para todos nuestros ejemplos hasta ahora.
- También podemos elegir la opción de Gallery y nos mostrará la pantalla de Activity que nos muestra al inicio de una aplicación nueva

File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help — Aplicacion Hello Word - AndroidManifest.xml [Aplicacion_Hello_Word.app]

AplicacionHelloWord 〉 app

app | Nexus 6 API 30

Android

activity_main.xml | MainActivity.java | AndroidManifest.xml

```
"1.0" encoding="utf-8"?>
s:android="http://schemas.android.com/apk/res/android"
om.example.aplicacionhelloword">

on
d:allowBackup="true"
d:icon="@mipmap/ic_launcher"
d:label="Aplicacion Hello Word"
d:roundIcon="@mipmap/ic_launcher_round"
d:supportsRtl="true"
d:theme="@style/Theme.AplicacionHelloWord">
ity
droid:name=".MainActivity"
droid:exported="true">
ntent-filter>
"android.intent.action.MAIN" />

e="android.intent.category.LAUNCHER" />
```

manifest

Text | Merged Manifest

- app
  - ma
  - java
  - java
  - res
  - res
  - Gradle

- New
- Add C++ to Module
- Cut — Ctrl+X
- Copy — Ctrl+C
- Copy Path...
- Paste — Ctrl+V
- Find Usages — Alt+F7
- Find in Files... — Ctrl+Mayús+F
- Replace in Files... — Ctrl+Mayús+R
- Analyze
- Refactor
- Add to Favorites
- Reformat Code — Ctrl+Alt+L
- Optimize Imports — Ctrl+Alt+O
- Run 'Tests in 'Aplicacion...' — Ctrl+Mayús+F10
- Debug 'Tests in 'Aplicacion...'
- More Run/Debug
- Open In
- Local History
- Reload from Disk
- Compare With... — Ctrl+D
- Open Module Settings — F4
- Load/Unload Modules...
- Mark Directory as
- Remove BOM
- Create Gist...
- Convert Java File to Kotlin File — Ctrl+Alt+Mayús+K

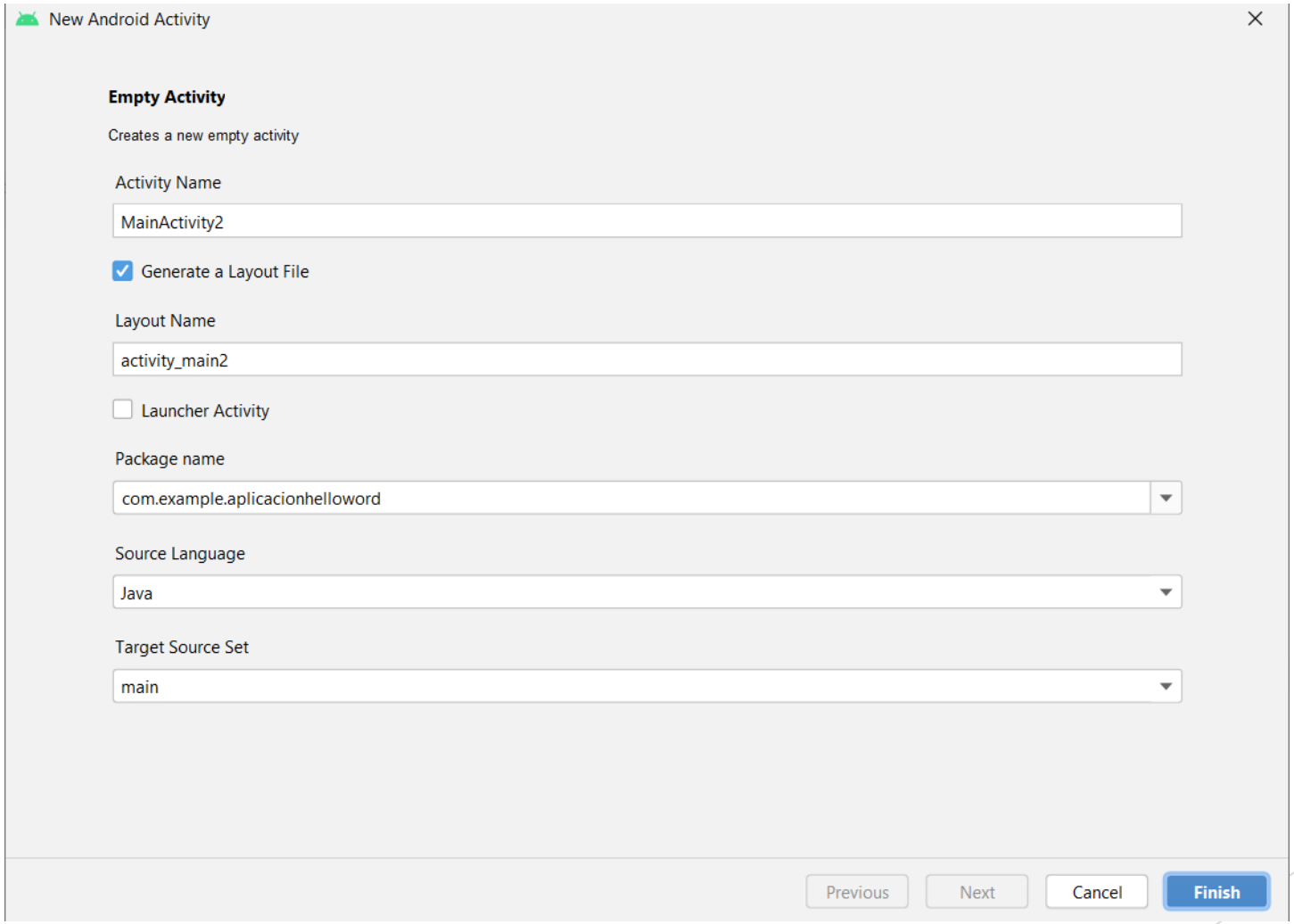
- Module
- Android Resource File
- Android Resource Directory
- Sample Data Directory
- File
- Scratch File — Ctrl+Alt+Mayús+Insertar
- Directory
- C++ Class
- C/C++ Source File
- C/C++ Header File
- Image Asset
- Vector Asset
- Kotlin Script
- Kotlin Worksheet
- Activity
- Fragment
- Folder
- Service
- UiComponent
- Automotive
- XML
- Wear
- AIDL
- Widget
- Google
- Compose
- Other
- EditorConfig File
- Resource Bundle

- Gallery...
- Android TV Blank Activity
- Basic Activity
- Bottom Navigation Activity
- Empty Activity
- Fragment + ViewModel
- Fullscreen Activity
- Login Activity
- Navigation Drawer Activity
- Primary/Detail Flow
- Responsive Activity
- Scrolling Activity
- Settings Activity
- Tabbed Activity

TODO | Problems | Terminal | Build | Logcat | Profiler | App Inspection | Run | Debug | Event Log | Layout Inspector

Activity secundario III

- En cualquiera de los dos casos, la siguiente pantalla que nos aparece es



Activity secundario IV

- En esta pantalla tendremos que poner un nombre al Activity. Lo ideal es poner un nombre que describa la función que va a tener o realizar y terminado siempre con la palabra Activity. De esta manera será más sencillo identificarlas.
- En este caso por ejemplo vamos a llamarlo SecundarioActivity.
- Se creará también el archivo .XML que será la interfaz de usuario de esta Activity.
- También tenemos la opción de asignar a este Activity que se el principal o inicial. No es recomendable marcar esta opción, ya que podría crear conflictos de programación porque ya tenemos otro Activity inicial
- En la siguiente pantalla veremos que se nos han creado los dos nuevos archivos.

AplicacionHelloWord > app > src > main > java > com > example > aplicacionhelloword

Project

Resource Manager

Android

- app
  - manifests
    - AndroidManifest.xml
  - java
    - com.example.aplicacionhelloword
      - MainActivity
      - SecundarioActivity
    - com.example.aplicacionhelloword (androidTest)
    - com.example.aplicacionhelloword (test)
  - java (generated)
  - res
    - drawable
    - layout
      - activity_main.xml
      - activity_secundario.xml
    - mipmap
    - values
  - res (generated)
- Gradle Scripts

Activity secundario V

- Si nos fijamos en los archivos MainActivity y el SecundarioActivity los dos llaman al método setContentView.
- Este método lo que hace es cargar en la pantalla del dispositivo la intefaz de usuario de cada uno de los Activity.
- Como ya os habréis dado cuenta, para referenciar al fichero XML, utilizamos un fichero estático, que es el fichero R que contiene todas las referencias a las variables y a los ficheros que tenemos en nuestro directorio de recursos (la carpeta res)
- Es un archivo que genera automáticamente AndroidStudio cada vez que creamos una nueva aplicación.

Activity secundario VI Actividad

- BUSCAR EL FICHERO R DENTRO DE NUESTRA APLICACIÓN.
- UNA VEZ LO HAYAS ENCONTRADO, BUSCA DENTRO DE ÉL EL BLOQUE ESTÁTICO LLAMADO LAYOUT.
- ¿HAY ALGO QUE TE RESULTE CONOCIDO DENTRO DE ESTE BLOQUE?
- ¿QUÉ OCURRE SI BORRAMOS UNA DE LAS LÍNEAS DE ESTE BLOQUE?
- ¿ME PRODRÍAS EXPLICAR QUE SIGNIFICA ENTONCES R.LAYOUT.ACTIVITY_SECUNDARIO?
- ESPERO VUESTRAS RESPUESTAS

Introducción a eventos.

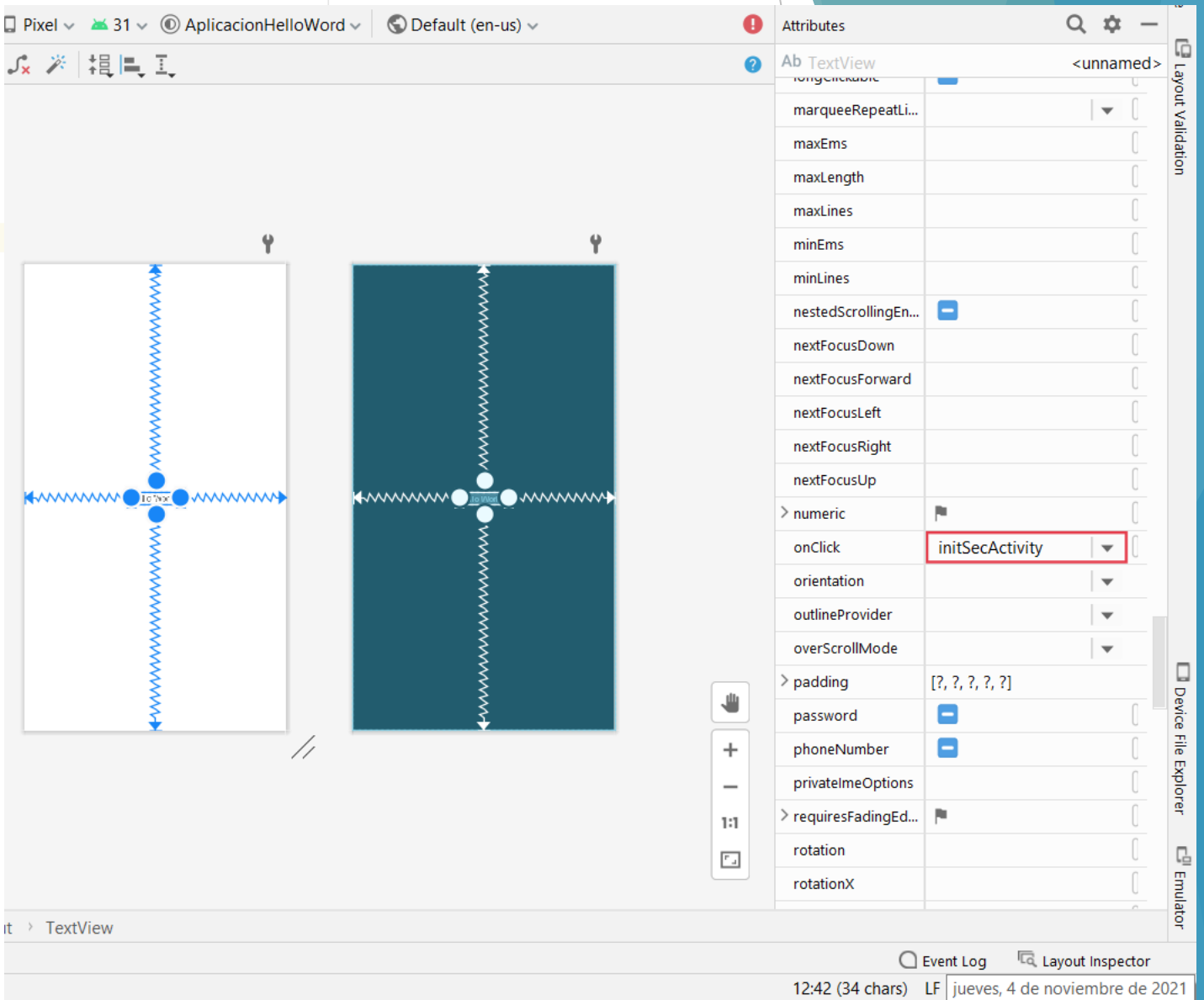
Evento onClick

Introducción a eventos I

- Para poder navegar por varias pantallas dentro de nuestra aplicación necesitaremos configurar eventos.
- Hay varias formas de poder configurarlo pero vamos a utilizar la más sencilla y que ya hemos realizado en algunos de nuestros ejemplos.
- Para ello nos iremos a nuestro layout principal y de nuestro ejemplo HelloWord.
- Dentro de ese archivo XML solo tenemos declarado un TextView en el que aparece el texto de Hola Mundo.
- El atributo que vamos a configurar es el atributo onClick de nuestro TextView.
- El valor que le vamos a dar es initSecActivity que hará referencia a nuestro activity secundario

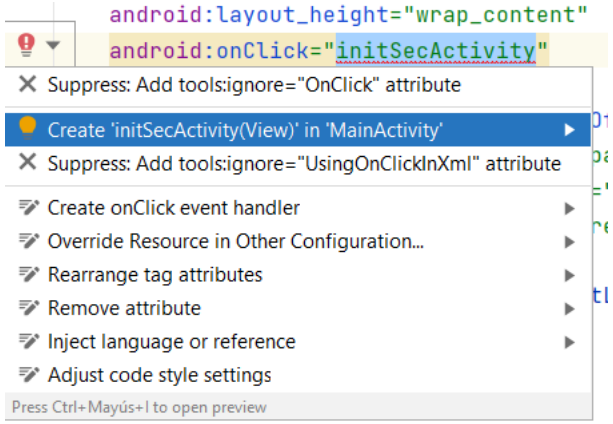
activity_main.xml | MainActivity.java | AndroidManifest.xml | activity_secundario.xml | SecundarioActivity.java

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      tools:context=".MainActivity">
8
9      <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:onClick="initSecActivity"
13         android:text="Hello World!"
14         app:layout_constraintBottom_toBottomOf="parent"
15         app:layout_constraintLeft_toLeftOf="parent"
16         app:layout_constraintRight_toRightOf="parent"
17         app:layout_constraintTop_toTopOf="parent" />
18
19 </androidx.constraintlayout.widget.ConstraintLayout>
```



Introducción a eventos II

- Como hemos visto en las imágenes lo que definimos en los atributos del entorno gráfico, aparecerá en el programa XML.
- El valor que aparece en onClick debe ser el nombre de un método que exista en el archivo MainActivity. Como en este caso no aparece, nos lo marca como un error.
- Si pulsamos en la bombilla amarilla que aparece, automáticamente nos crea un método en el archivo MainActivity con la siguiente estructura.




```
    public void initSecActivity(View view) {
    }
}
```

Introducción a eventos III

- Para comprobar que al ejecutar la aplicación entra en el método vamos a utilizar una nueva forma.
- En lugar de utilizar el log que utilizamos en el ciclo de vida, vamos a usar la función TOAST. Esta función lo que hace es mostrar un mensaje emergente durante unos segundos en la pantalla del usuario.
- Vamos a crear uno nuevo (opción que aparece en autocompletar).

```
}
    Toast                                   Create a new Toast
    c Toast android.widget
    Press Ctrl+. to choose the selected (or first) suggestion and insert a dot afterwards  Next Tip
    Toas
}
```

Introducción a eventos IV

- This nos va a definir el contexto en el que estamos lanzando este mensaje. Como estamos dentro del MainActivity es suficiente con poner la palabra reservada this porque hará referencia al propio Activity.
- En el segundo parámetro vamos a escribir el texto: “Has hecho click en el texto” que es el mensaje emergente que nos mostraría.
- El último parámetro define el número de segundos que va a aparecer el mensaje en la pantalla. Tenemos dos opciones:
  - Toast.LENGTH_SHORT: que lo mostrará dos segundos.
  - Toast.LENGTH_LONG: que lo mostrará cuatro segundos.
- Nos quedaría algo así:

```
    public void initSecActivity(View view) {
        Toast.makeText( context: this,  text: "Has hecho click en el texto", Toast.LENGTH_LONG).show();
    }
}
```

Introducción a eventos V

- El resultado es el siguiente:

Introducción a eventos VI

- Hay algunas formas más de hacer este proceso:
  - Definiendo un identificador para el componente, utilizando el atributo id del elemento.
  - Después creamos una variable del mismo tipo, en este caso TextView y lo asociamos al elemento visual.
  - Una vez realizado lo anterior, debemos crear un escuchador de eventos.
  - Todos estos pasos quedarían así:

```
<TextView
    android:id="@+id/txtEvento"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

```
public class MainActivity extends AppCompatActivity {
    private int i;
    TextView texto;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onCreate");
        i = 0;
        texto = (TextView) findViewById(R.id.txtEvento);
        texto.setOnClickListener();
    }
```

Introducción a eventos VI

En este punto podemos elegir dos formas diferentes de realizar el proceso:

- La primera es gestionar, en el propio escuchador, una clase anónima que definiría el evento click.
- Dentro del escuchador se generaría un método onClick similar al que hemos realizado antes.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Log.i( tag: "TAG:cicloVida", msg: "Ciclovida:onCreate");
    i = 0;
    texto = (TextView) findViewById(R.id.txtEvento);
    texto.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            Toast.makeText( context: MainActivity.this,  text: "Has hecho click en el escuchador", Toast.LENGTH_LONG).show();
        }
    });
}
```

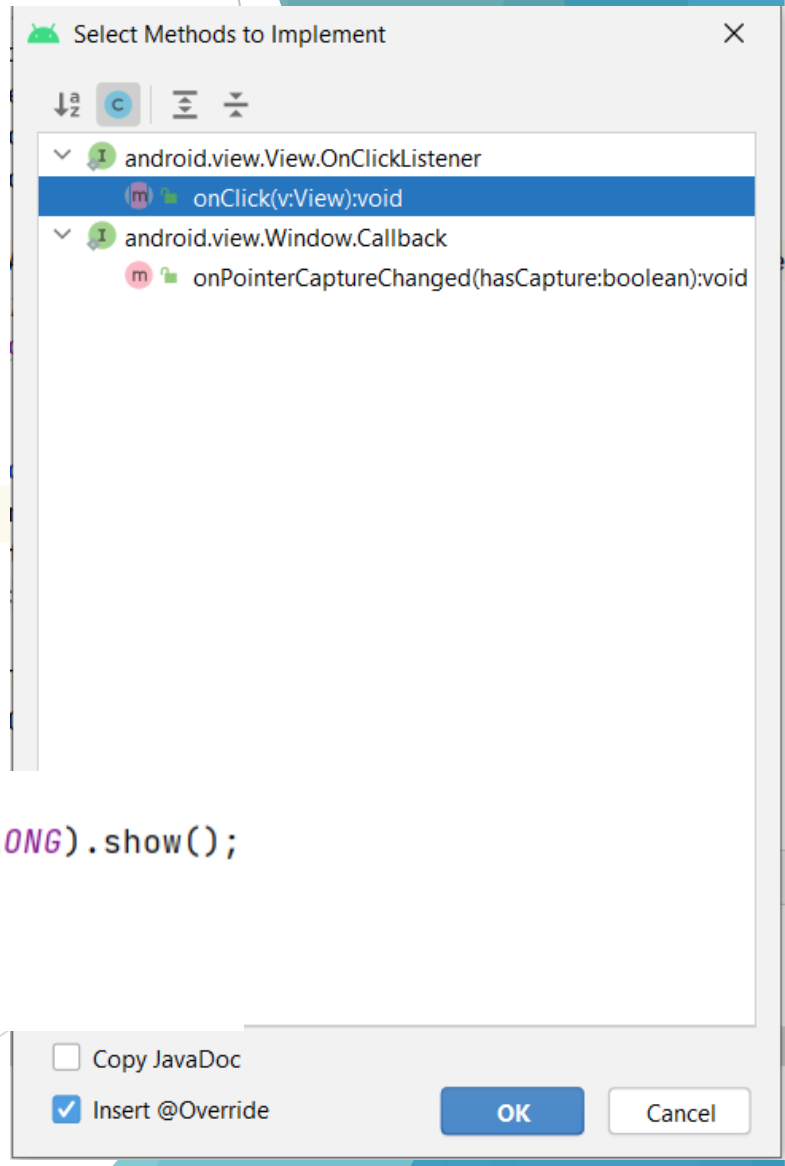
Introducción a eventos VII

- La segunda es declarar en la clase la opción de gestionar eventos click.
- Quedaría de la siguiente manera:

```
texto = (TextView) findViewById(R.id.txtEvento);
texto.setOnClickListener(this);
```

Cast parameter to 'android.view.View.OnClickListener'
Make 'MainActivity' implement 'android.view.View.OnClickListener'
Press Ctrl+Mayús+I to open preview

```
verride

    public void onClick(View view) {
        Toast.makeText( context: this,  text: "Has hecho click en el texto", Toast.LENGTH_LONG).show();

    }
}
```



Las Intents.

Evento onClick

Intent

- Las INTENT son unos componentes que nos permiten iniciar nuevas actividades
- Tenemos dos tipos
  - Explícitas: que son activity que existen en nuestra aplicación
  - Implícitas: que son activity que existen en otra aplicación.
- Comenzaremos las intent explícitas.

Intent explícitas I

- Las intents explícitas especifican qué aplicación las administrará, ya sea incluyendo el nombre del paquete de la app de destino o el nombre de clase del componente completamente calificado.
- Normalmente, el usuario usa una intent explícita para iniciar un componente en su propia aplicación porque conoce el nombre de clase de la actividad o el servicio que desea iniciar.
- Por ejemplo, puedes utilizarla para iniciar una actividad nueva en respuesta a una acción del usuario o iniciar un servicio para descargar un archivo en segundo plano.

Intent explícitas II

- Vamos a crear un proyecto nuevo que vamos a llamar IntentExplicita.
- Vamos a crear un segundo Activity que llamaremos DatosActivity.
- En el activity-main vamos a generar un botón que vamos a llamar Iniciar Datos Activity. En este botón vamos a asignar un método al onClick que se va a llamar InicarActivity.
- Ahora vamos a crear el método IniciarActivity en el fichero MainActivity.
- Cuando nosotros hagamos click en el botón se lanzará el método IniciarActivity.
- Nuestro método contendrá el siguiente código

Intent explícitas III

```
16
17      public void IniciarActivity(View view) {
18          Intent intentDatos = new Intent( packageContext: this, DatosActivity.class);
19          startActivity(intentDatos);
20      }
21  }
```

- La primera línea del método es para crear una nueva intent. Si no estuviera la clase Intent, habría que importarla.
  - Dentro del constructor indicamos en primer lugar el contexto, que hace referencia al Activity sobre el que estamos trabajando. Como en este caso estamos trabajando sobre MainActivity, podemos poner la palabra reservada this.
  - En el segundo parámetro, pondremos la clase java que queremos lanzar, en nuestro caso DatosActivity.class.
- La segunda línea es la que nos indica la ejecución de Intent. Incluimos como parámetro la intent que hemos definido en la línea anterior.

Intent explícitas IV

- Para comprobar que funciona correctamente, podemos modificar el Activity secundario y poner algún texto significativo (por ejemplo: estas en DatosActivity) para distinguir si está accediendo a él cuando pulsamos el botón del Activity principal.
- Una vez modificado, podemos lanzar la aplicación y comprobamos que funciona correctamente.

Intent explícitas V

- A través de las Intent también podemos pasar parámetros.
- Se añadirían entre la línea de declaración de la intent y el lanzamiento.
- Se pondrían tantas líneas de código como parámetros queramos declarar.
- Si por ejemplo queremos pasar por parámetro el año en el que estamos o un número pondríamos los siguiente:
  - intentDatos.putExtra (“numero”, 1); o intentDatos.putExtra (“nombre”, “Laura”);
  - La estructura siempre es igual: primero el nombre de la variable que queremos pasar como parámetro al siguiente activity y después su valor.

Intent explícitas VI

- Para recuperar esos parámetros en el siguiente activity tendríamos que hacer lo siguiente:
  - Declaramos un Bundle que es un conjunto de variables que podemos recuperar a través del nombre con el que hemos definido los parámetros. En el caso de nuestro ejemplo número y nombre.

    ```
    Bundle extras = getIntent().getExtras();
    ```

  - Para obtener valores tendremos que utilizar la función getIntent.
  - Después recuperaremos nuestros datos. Para ello pondremos las siguientes líneas de código:

    ```
    int n = extras.getInt("numero");
    String s = extras.getString ("nombre");
    ```

  Para mostrarlo por pantalla usaremos un TOAST

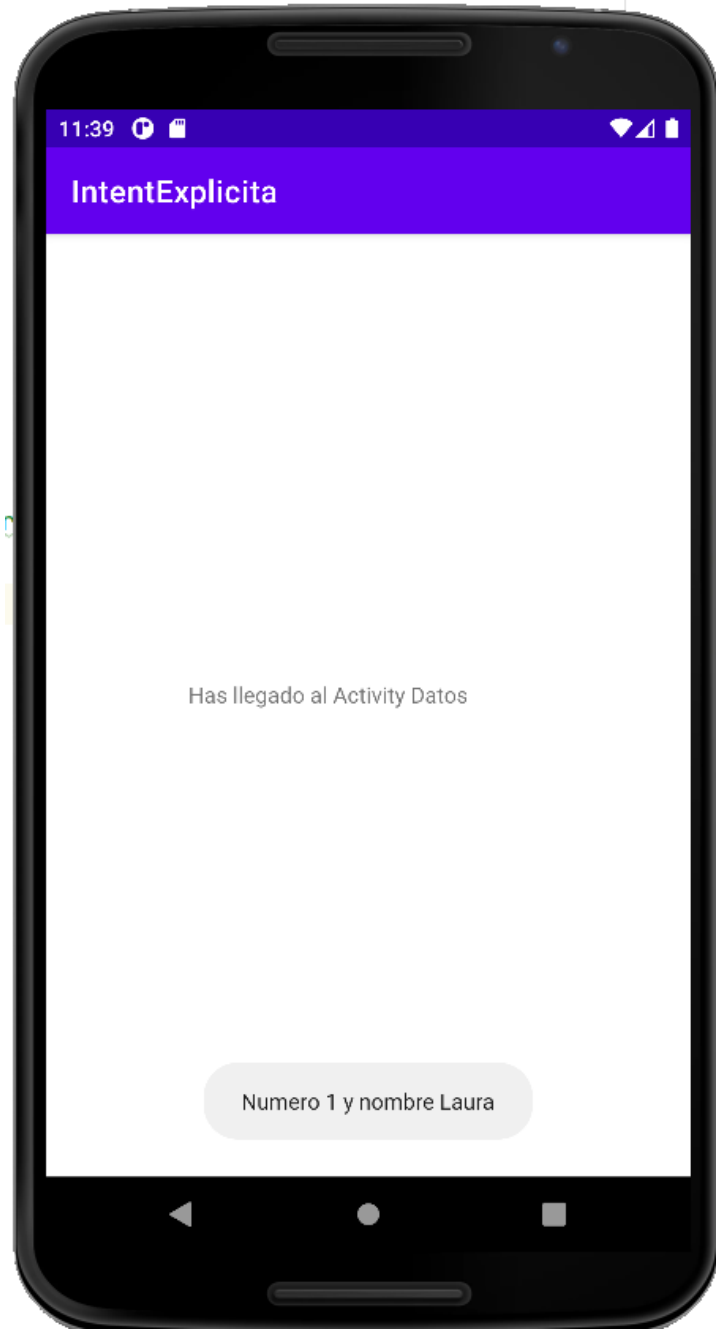
El código de programación sería

activity_main.xml | MainActivity.java | activity_datos.xml | DatosActivity.java

```java
package com.example.intentexplicita;

import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void IniciarActivity(View view) {
        Intent intentDatos = new Intent( packageContext: this, DatosActivity.class);
        intentDatos.putExtra ( name: "numero",  value: 1);
        intentDatos.putExtra ( name: "nombre",  value: "Laura");

        startActivity(intentDatos);
    }
}
```

activity_main.xml | MainActivity.java | activity_datos.xml | DatosActivity.java

```java
package com.example.intentexplicita;

import ...

public class DatosActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_datos);
        Bundle extras = getIntent().getExtras();
        int n = extras.getInt( key: "numero");
        String s = extras.getString( key: "nombre");
        Toast.makeText( context: this,  text: "Numero "+n+" y nombre "+s, Toast.LENGTH_LONG).show();
    }
}
```



El resultado visual

Probad a…

- Sobre el código del PuzleBotones:
  1. Añadid un segundo activity.
  2. En el principal colocad un botón que cuando sea pulsado pase al segundo activity.
  3. El segundo activity debe mostrar un TextView en el que aparezca la cantidad de pulsaciones que se han hecho en el puzle.

Intent implícitas I

- Las Intent implícitas no lanzan un activity concreto si no que se lanza una acción genérica.
- Permite llamar al S.O. o a cualquier otra aplicación para que resuelva nuestra necesidad. Por ejemplo reproducir un archivo de sonido o un vídeo, llamadas o direcciones en un mapa y muchas otras opciones más.
- Nos van a ahorrar mucha implementación, ya que no siempre es necesario realizar todas esas funcionalidades desde cero.
- Para verlo más claro vamos a realizar un ejemplo. En este caso vamos a realizar una alarma en nuestro smartphone.

Intent implícitas II

- Creamos un método que se llama createAlarm que tendrá tres parámetros:
  - El primero es el parámetro message que será el nombre que aparezca cuando se active la alarma.
  - Los siguientes son la hora y los minutos en los que queremos que se active la alarma.
- En la siguiente línea de código aparece el startActivity que es el método para llamar a las INTENT
- Un ejemplo de código sería el siguiente:

Intent implícitas III

- Para ello vamos a escribir el siguiente código:

```
public void createAlarm(String message, int hour, int minutes) {
    Intent intent = new Intent(AlarmClock.ACTION_SET_ALARM)
            .putExtra(AlarmClock.EXTRA_MESSAGE, message)
            .putExtra(AlarmClock.EXTRA_HOUR, hour)
            .putExtra(AlarmClock.EXTRA_MINUTES, minutes);
      startActivity(intent);

}
```

Para llamar al intent lo hacemos igual que lo hacíamos con los explícitos con el método startActivity.

Intent implícitas IV

- Para poder utilizar este código en uno de nuestras aplicaciones, vamos a crear un botón en el activityMain de nuestro Main principal.
- A este botón le vamos a llamar Crear Alarma a las 12:15.
- A este botón le vamos a asociar un evento onClick con un método crearAlarma que crearemos en nuestro fichero MainActivity.
- En ese método vamos a poner el código que hemos explicado en las diapositivas anteriores.
- La programación quedará de la siguiente manera:

Intent implícitas V

```
public void crearAlarma(View view) {
    Intent intent = new Intent(AlarmClock.ACTION_SET_ALARM)
            .putextra(AlarmClock.EXTRA_MESSAGE, value: "Ir al trabajo")
            .putExtra(AlarmClock.EXTRA_HOUR, value: 11)
            .putExtra(AlarmClock.EXTRA_MINUTES, value: 15);
    startActivity(intent);
}

}
```

- Recordad que si no reconoce los métodos, tendremos que importar la librería que necesitemos.

Intent implícitas VI

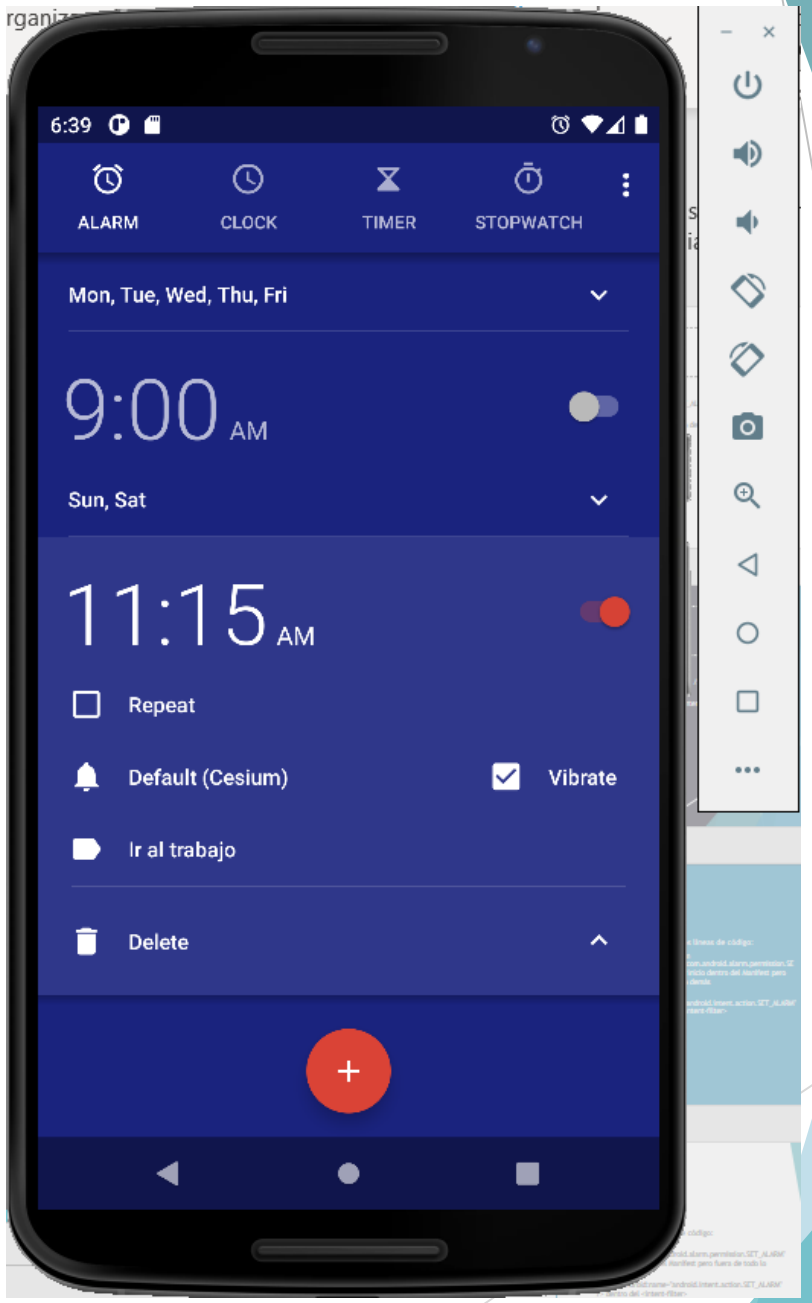
- Seguramente al pulsar el botón no aparezca nada. Eso es debido a que nos falta aún una línea de código que añadir en el AndroidManifest.
- Este archivo nos quedaría así:

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.miprimeraalarma">
    <uses-permission android:name="com.android.alarm.permission.SET_ALARM" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="MiPrimeraAlarma"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MiPrimeraAlarma">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Intent implícitas VII

- Hemos añadido esta línea de código:
  - <uses-permission android:name="com.android.alarm.permission.SET_ALARM" /> al inicio dentro del Manifest pero fuera de todo lo demás
- Una vez puestas la línea de código el resultado debería ser que cuando se pulsa el botón que hemos creado, aparece la central de alarmas del dispositivo, con nuestra alarma creada.

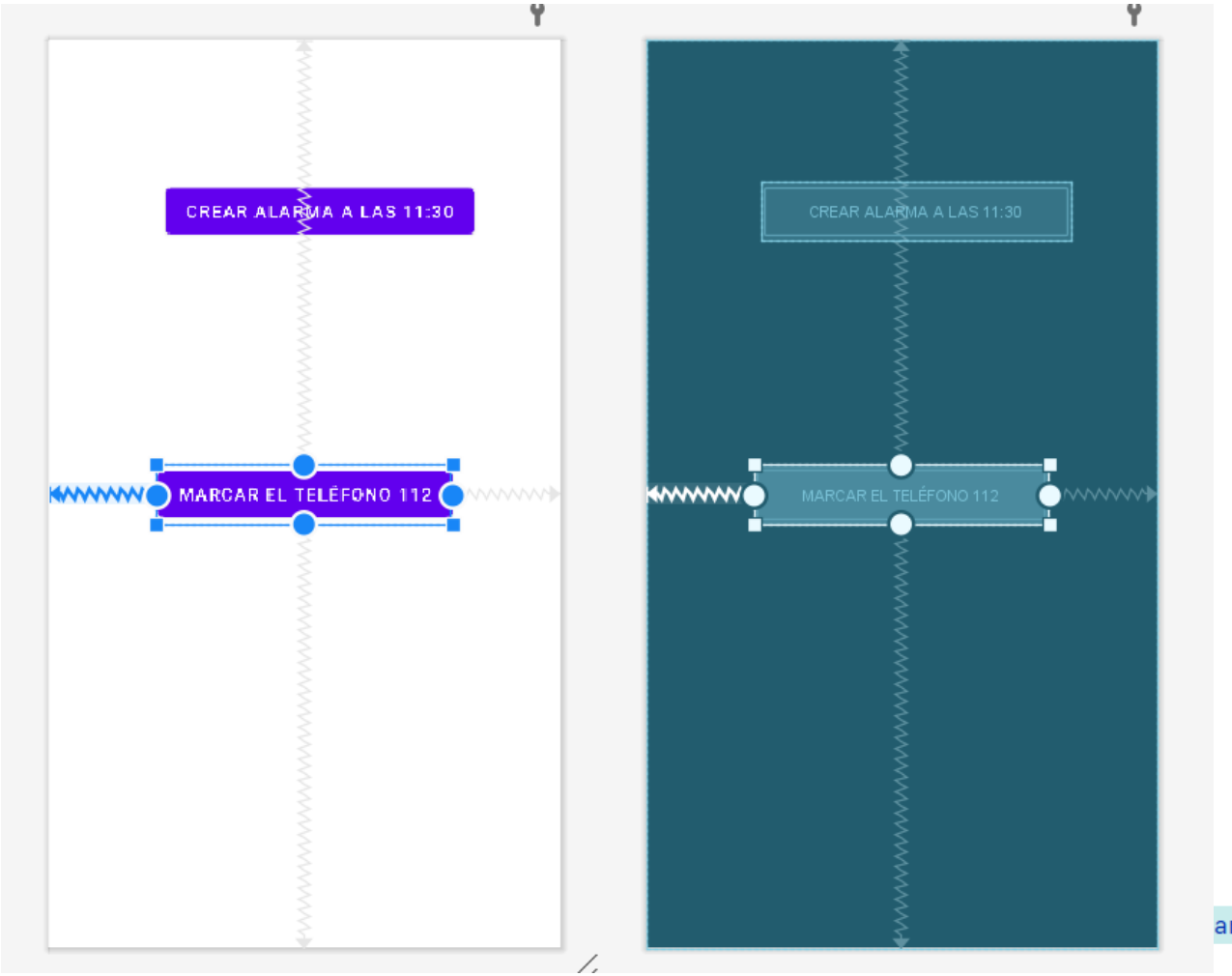
Intent implícitas VIII



Intent implícitas IX. Ejemplo para realizar en clase

- VAMOS A IMPLEMENTAR UNA APLICACIÓN PARA PODER REALIZAR UNA LLAMADA DE TELÉFONO.
- VAMOS A TENER DOS OPCIONES ACTION_DIAL Y ACTION_CALL.
- ¿CUAL ES LA DIFERENCIA ENTRE LAS DOS OPCIONES?
- HAZ UN EJEMPLO UTILIZANDO ACTION_DIAL
- ¿QUE ES UN FORMATO URI?
- CUANDO TENGAIS HECHO EL EJEMPLO AVISADME PARA PODER VERLO

Activity_Main




```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="crearAlarma"
    android:text="Crear Alarma a las 11:30"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.574"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.166" />

<Button
    android:id="@+id/button2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="marcarTlfEmergencia"
    android:text="Marcar el teléfono 112"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
androidx.constraintlayout.widget.ConstraintLayout>
```

AndroidManifest

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.miprimeraalarma">
    <uses-permission android:name="com.android.alarm.permission.SET_ALARM" />
    <uses-permission android:name="android.permission.CALL_PHONE" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="MiPrimeraAlarma"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MiPrimeraAlarma">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

MainActivity

```java
import android.net.Uri;
import android.os.Bundle;
import android.provider.AlarmClock;
import android.view.View;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void crearAlarma(View view) {
         Intent intent = new Intent(AlarmClock.ACTION_SET_ALARM)
              .putextra(AlarmClock.EXTRA_MESSAGE,  value: "Ir al trabajo")
              .putExtra(AlarmClock.EXTRA_HOUR,  value: 11)
              .putExtra(AlarmClock.EXTRA_MINUTES,  value: 15);
        startActivity(intent);
    }

    public void marcarTlfEmergencia(View view) {
        Intent intent = new Intent(Intent.ACTION_DIAL);
        intent.setData(Uri.parse("tel:" + 112));
        startActivity(intent);
    }

}
```

Intent implícitas X

- La otra opción de uso de las Intent implícitas es ser la aplicación que de respuesta a la solicitud de la Intent.
- Es decir y siguiendo con nuestro primer ejemplo, que nuestra aplicación sea una aplicación de gestión de alarmas.
- Para ello creamos un nuevo Activity que le vamos a llamar gestionAlarmasActivity de tipo empty.
- Para comprobar como funciona, en este activity que hemos creado, solo vamos a poner Soy el gestor de alarmas.
- Esta será nuestra aplicación, aunque como vemos no está creada como tal.

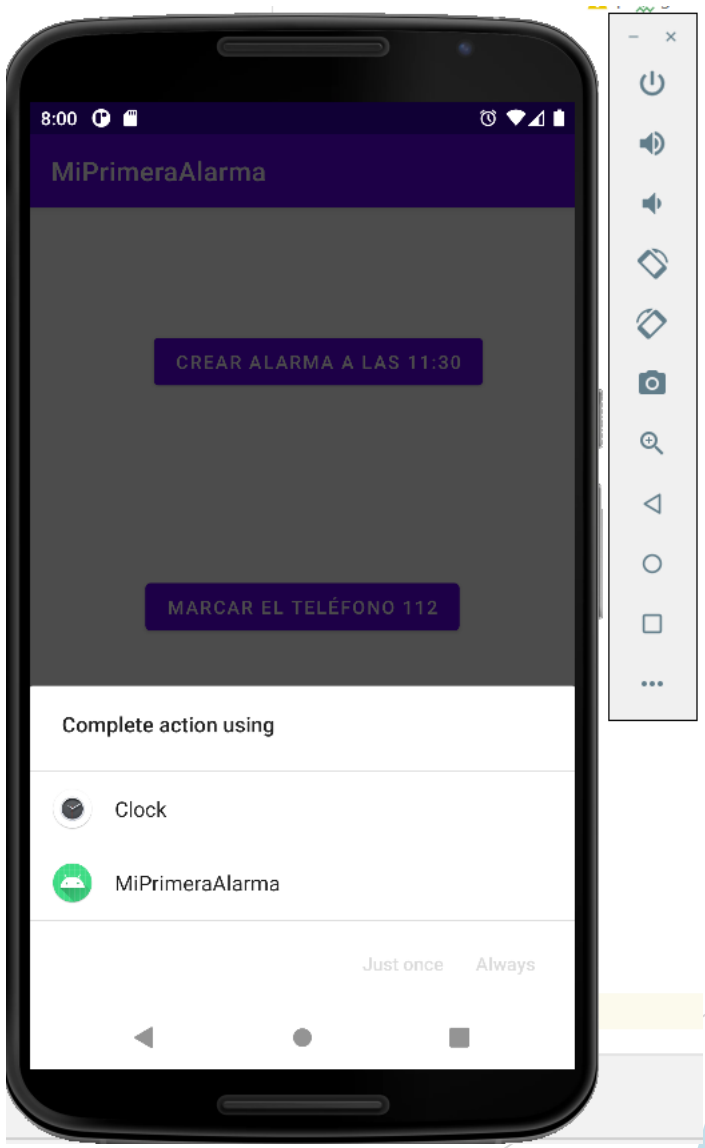
Intent implícitas XI

- Para que nuestra aplicación aparezca el S.O. como una aplicación que da respuesta a la creación de alarmas, vamos a tener que modificar nuestro AndroidManifest.
- En este archivo añadiremos un <intent-filter> que le indicará al S.O. que nosotros podemos resolver esa acción.
- El fichero quedaría así:

```
<activity
    android:name=".GestionAlarmasActivity"
    android:exported="true" >
    <intent-filter>
        <action android:name="android.intent.action.SET_ALARM" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

Intent implícitas XI

- Una vez realizados todos estos pasos y si damos a ejecutar la práctica, nos aparecerán dos opciones para gestionar las alarmas: una la del propio S.O. y otra la que acabamos de crear nosotros.
- El resultado sería:



Diseño de interfaces de usuarios

Layout

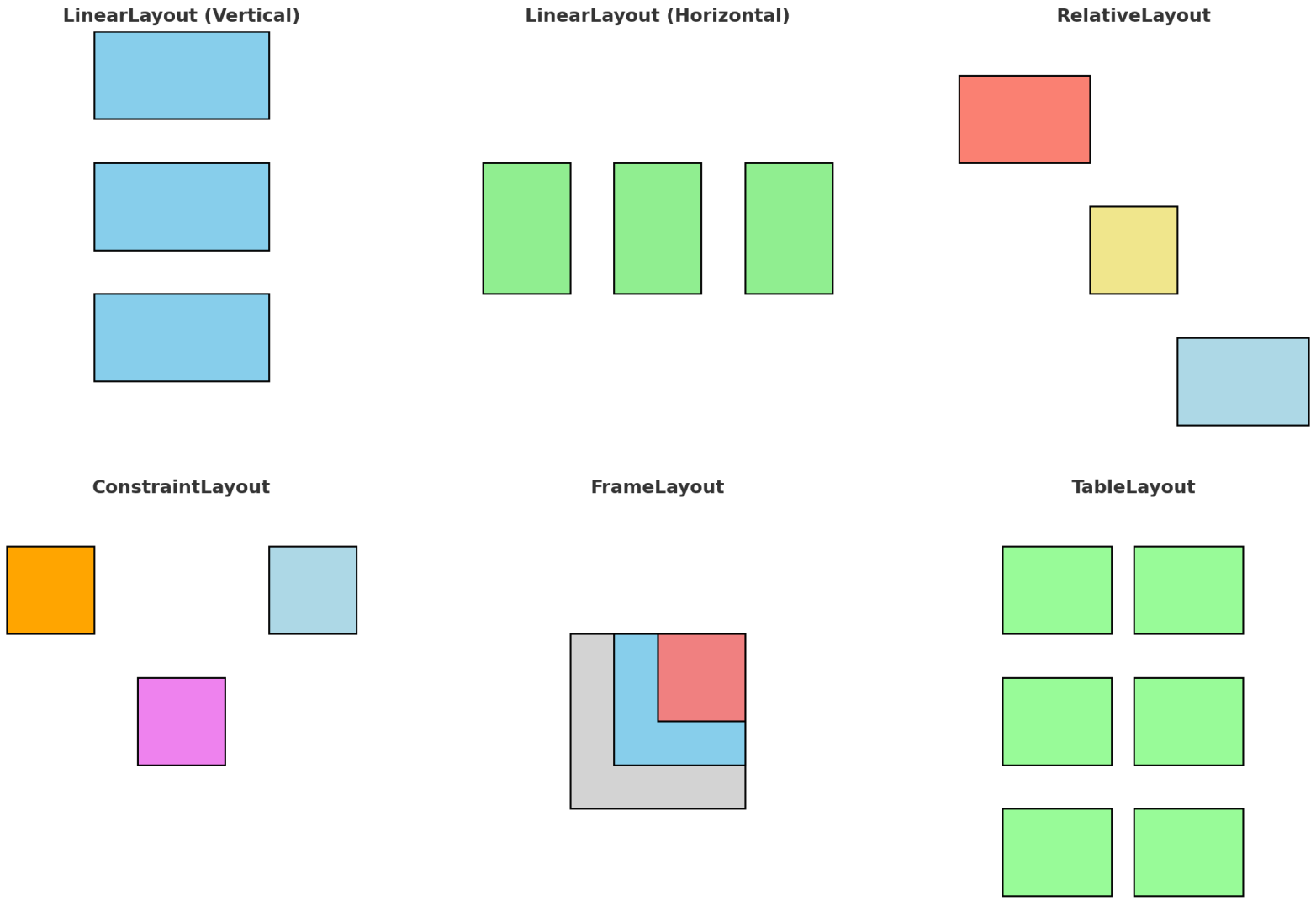
Tipos de Layout

- LinearLayout es uno de los Layout más utilizado en la práctica. Distribuye los elementos uno detrás de otro, bien de forma horizontal o vertical.
- TableLayout distribuye los elementos de forma tabular. Se utiliza la etiqueta <TableRow> cada vez que queremos insertar una nueva línea. Existe un GridLayout que añade algunas opciones a partir del nivel 11 (Android ICS 4.0) o superior.
- RelativeLayout permite comenzar a situar los elementos en cualquiera de los cuatro lados del contenedor e ir añadiendo nuevos elementos pegados a estos.
- ConstraintLayout Versión más flexible y eficiente de RealiveLayout.
- FrameLayout posiciona las vistas usando todo el contenedor, sin distribuirlas espacialmente. Este Layout suele utilizarse cuando queremos que varias vistas ocupen un mismo lugar. Podemos hacer que solo una sea visible, o superponerlas.Para modificar la visibilidad de un elemento utilizaremos la propiedad visibility.
- AbsoluteLayout permite indicar las coordenadas (x,y) donde queremos que se visualice cada elemento. No es recomendable utilizar este tipo de Layout. La aplicación que estamos diseñando tiene que visualizarse correctamente en dispositivos con cualquier tamaño de pantalla. Para conseguir esto, no es una buena idea trabajar con coordenadas absolutas. De hecho, este tipo de Layout ha sido marcado como obsoleto.

Tipos de Layout. Cuando usarlos.

- Si tienes dudas sobre cuando usar cada layout usa la siguiente tabla:
  - LinearLayout: Diseños muy sencillos.
  - RelativeLayout: Nunca, hay una nueva alternativa
  - ConstraintLayout: Usar por defecto.
  - TableLayout: Distribuciones en celdas.
  - FrameLayout: Varias vistas superpuestas.
  - AbsoluteLayout: Nunca. Aunque está bien conocerlo por si acaso.

Tipos de Layout. Visualmente.



LinearLayout I

- LinearLayout es un diseño básico para crear las interfaces de usuario de nuestras aplicaciones.
- Coloca las vistas una a continuación de otra, sin superponerlas nunca. Es decir, las alinea.
- Esta alineación puede ser vertical u horizontal. O sea, que LinearLayout sirve para colocar vistas en una misma fila o columna, pero no ambas cosas a la vez.
- Las vistas se colocan en el mismo orden en el que se agregan al diseño o en el que aparecen en el archivo XML.
- De forma predeterminada, LinearLayout tiende a establecer el mismo tamaño para cada vista que contiene, repartiendo el espacio disponible de forma equitativa entre todas ellas. Sin embargo, algunas propiedades de las vistas y ciertos parámetros que el propio LinearLayout añade permiten variar este comportamiento.

LinearLayout II

- Para poder probar como funciona, vamos a generar un proyecto nuevo y una vez generado vamos a entrar en el activity-main.xml o layout.
- Vamos a duplicar el TextView que tenemos para poder tener al menos dos elementos.
- Tenemos como texto Hello world pero podemos poner el texto que nos guste más.

LinearLayout III

- En ese archivo aparece en la segunda línea una etiqueta con el siguiente nombre <androidx.constraintlayout.widget.ConstraintLayout
- Esa etiqueta la vamos a modificar y vamos a poner en su lugar LinearLayout

activity_main.xml MainActivity.java

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <Line xmlns:android="http://schemas.android.com/apk/res/android"
3      androidx.appcompat.widget.LinearLayoutCompat
4      LinearLayout
5      com.google.android.material.progressindicator.LinearPro…
6      androidx.constraintlayout.widget.Guideline
7      com.google.android.material.circularreveal.CircularReve…
8      android.widget.inline.InlineContentView
9      com.google.android.material.internal.BaselineLayout
10     TwoLineListItem
11     Ctrl+Abajo and Ctrl+Arriba will move caret down and up in the editor  Next Tip
12         android:text="Hello World!"
13         app:layout_constraintBottom_toBottomOf="parent"
14         app:layout_constraintLeft_toLeftOf="parent"
15         app:layout_constraintRight_toRightOf="parent"
16         app:layout_constraintTop_toTopOf="parent" />
17
18 </Line>
```

LinearLayout IV

- Los elementos hijos de LinearLayout, es decir los elementos que están dentro de esta etiqueta, tienen una propiedad que es la propiedad android:layout_weight
- Esta propiedad nos permite definir las proporciones que queramos que ocupe ese elemento.
- Si por ejemplo le damos el valor 1 en cada uno de nuestros TextView de nuestro proyecto android:layout_weight="1", el aspecto de la interfaz de usuario sería la siguiente:

LinearLayout V

Muestra todos los elementos que estén dentro del LinearLayout uno al lado del otro, es decir en orientación horizontal y ocupando la misma porción de espacio.

Si queremos cambiar la orientación y que aparezcan en vertical, debemos utilizar la propiedad orientation que la ubicaremos en las propiedades de LinearLayout.

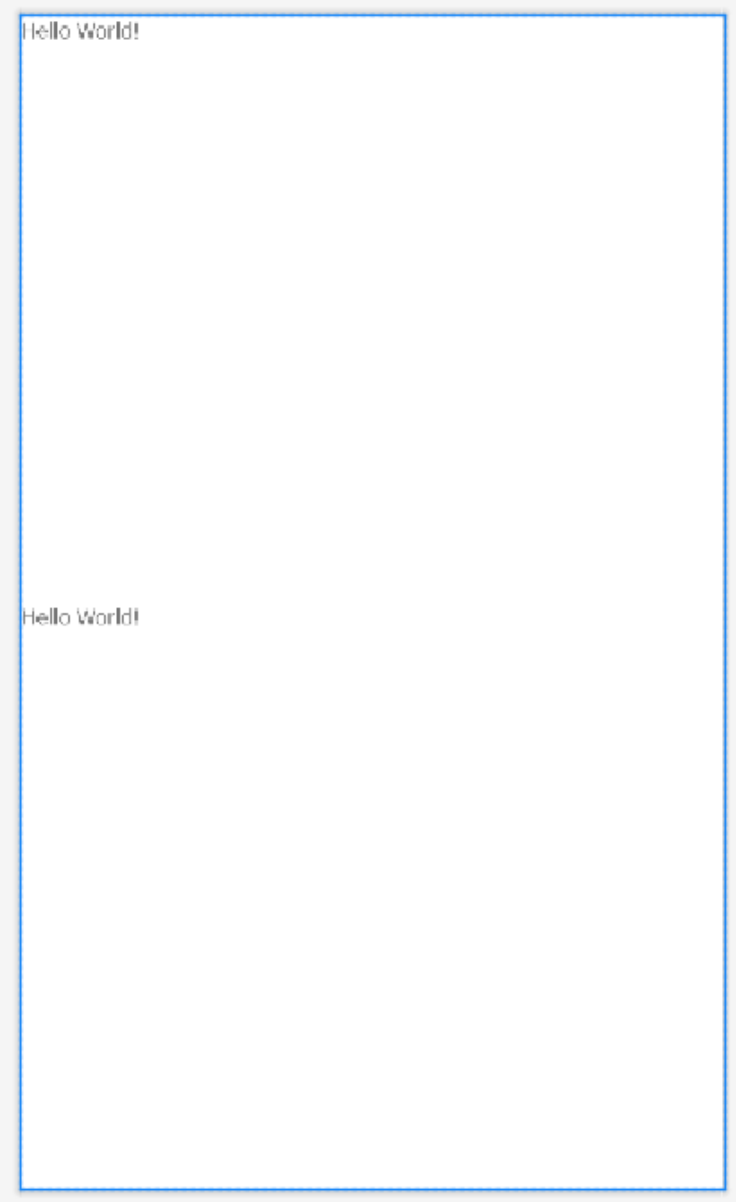
LinearLayout VI

- El código quedaría de la siguiente manera:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
tools:context=".MainActivity">
```

LinearLayout VII

- Y visualmente quedaría:



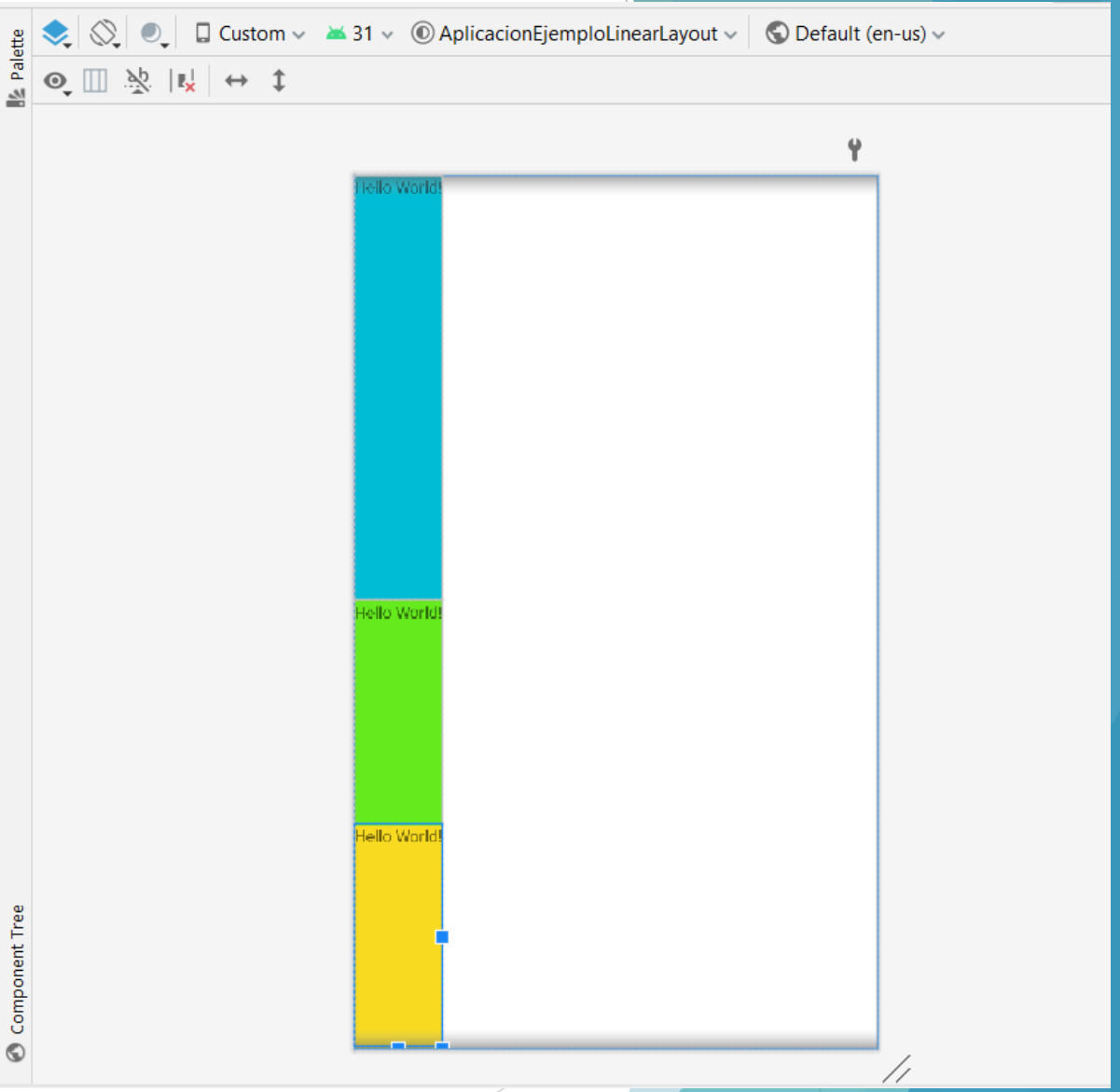
LinearLayout VIII

- En caso de que queramos que cada elemento ocupe un porcentaje diferente de la pantalla, deberíamos indicar en la propiedad weight un número diferente para indicarle el tamaño.
- Imaginemos que tenemos tres elementos TextView en formato vertical. Queremos que el primer elemento ocupe el 50% de la pantalla y los otros dos el 25% cada uno.
- ¿Cómo lo harías?

LinearLayout IX

- La solución es simple. Dividiremos mentalmente nuestra pantalla en 4 partes iguales. Al elemento que queremos darle el 50% le daremos 2 partes y a los otros dos les daremos una parte a cada uno haciendo un total de 4 partes.
- Para verlo mejor, podemos dar un color a cada elemento, utilizando la propiedad background y asignándole un color.
- Eso significa que nuestro código y nuestra interfaz quedarían de la siguiente manera:

```
<TextView
    android:background="#00BCD4"
    android:layout_weight="2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
<TextView
    android:background="#64EA1D"
    android:layout_weight="1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
<TextView
    android:background="#F8DB21"
    android:layout_weight="1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```



RelativeLayout I

- Es un grupo de vistas que muestra vistas secundarias en posiciones relativas.
- La posición de cada vista puede especificarse como relativa a elementos del mismo nivel (como a la izquierda de otra vista o por debajo de ella) o en posiciones relativas al área RelativeLayout superior (como alineada a la parte inferior, izquierda o central).
- Nos permite definir interfaz de usuario más complejas.
- Siempre se comienza a diseñar por la esquina superior izquierda.
- Vamos a hacer un ejemplo para entenderlo mejor.

RelativeLayout II

- Vamos a realizar un diseño lo más parecido a lo siguiente:

- Si empezamos por la esquina superior izquierda nos aparece una imagen que nos podemos descargar de una página web. Por ejemplo, en la página web https://fonts.google.com/icons?selected=Material+Icons están los iconos disponibles en la librería de Android.
- Si buscamos con la palabra camera, aparecerá el icono que nosotros necesitamos y lo descargamos.

RelativeLayout III

- Abrimos un nuevo proyecto que llamaremos relativeLayout
- Incluimos dentro de la carpeta res\drawable la imagen descargada. Para ello pulsaremos el botón derecho sobre la carpeta, le daremos a new y elegiremos la opción vector asset y elegiremos nuestra imagen descargada en local.
- Modificamos el archivo activity-main.xml. Para ello eliminaremos el TextView que crea automáticamente y modificamos el <androidx.constraintlayout.widget.ConstraintLayout y pondremos <RelativeLayout
- En el entorno visual insertamos un elemento ImageView y seleccionamos la imagen que hemos descargado.
- Cuando la imagen aparece, se nos sitúa en el punto mas arriba y más a la izquierda. Para poder moverla de posición tendremos que marcar los márgenes. Pero no aparecen de la misma manera que en ejemplos anteriores.

RelativeLayout IV

- Todos los elementos que vayamos insertando en un RelativeLayout van a guardar relación con respecto a la ubicación de otros elementos. Si no hay elementos previos, la relación la guarda con respecto a la pantalla.
- Al hacer esa relación con los bordes de la pantalla, aparecen en el código XML dos nuevas líneas de código que son:

```
android:layout_alignParentStart="true“
android:layout_alignParentTop="true"
```

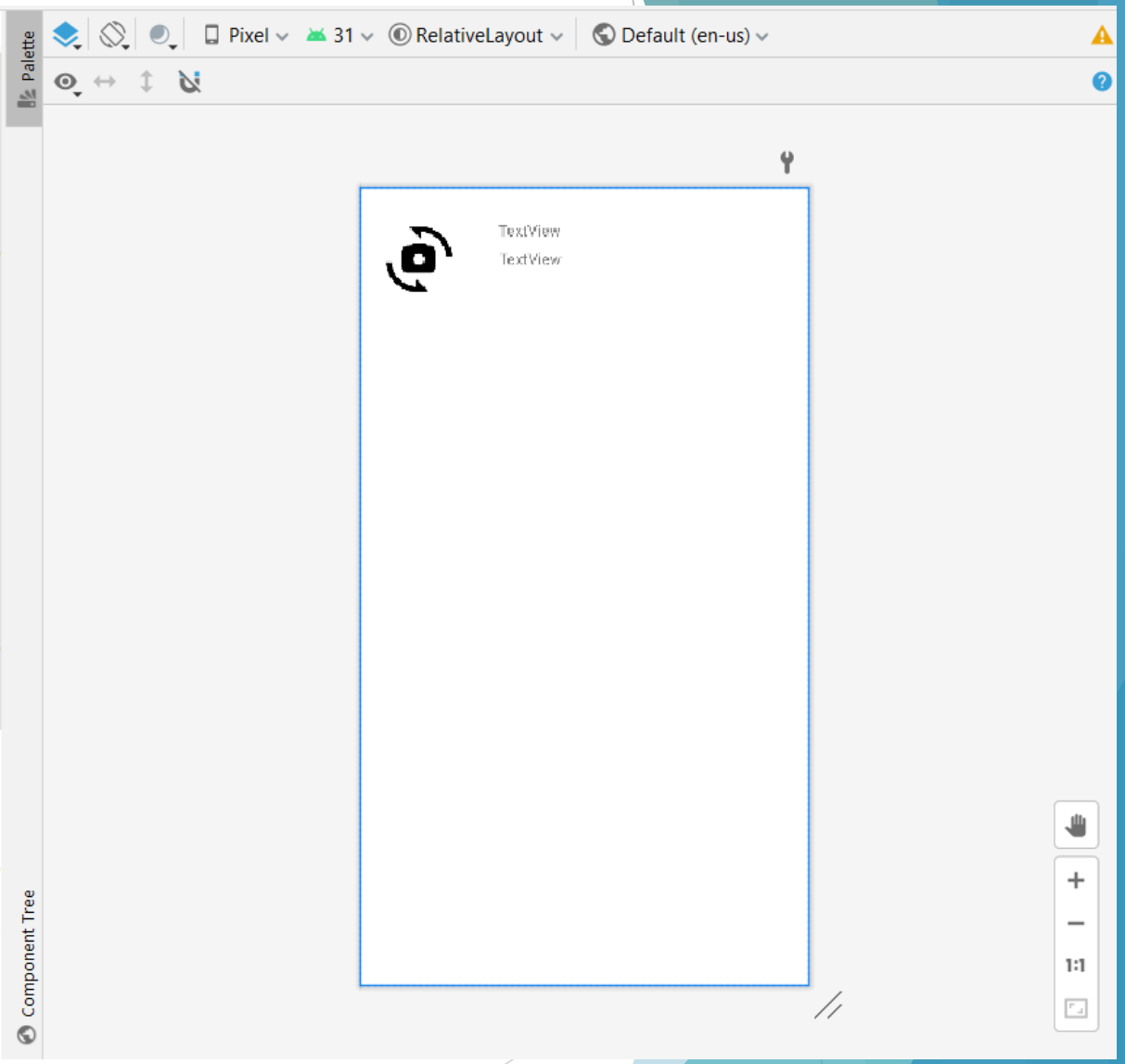
- Si pusiéramos la relación con los 4 bordes aparecerían dos líneas más:

```
android:layout_alignParentEnd="true"
android:layout_alignParentBottom="true“
```

RelativeLayout V

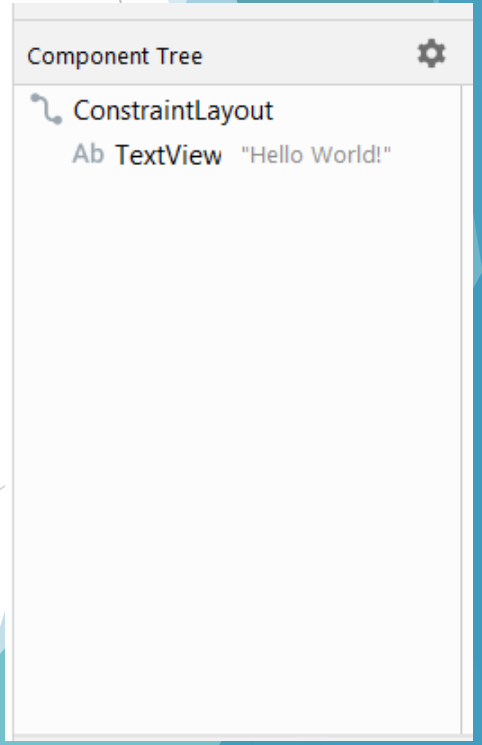
- Es muy importante que cada vez que añadamos un elemento le cambiemos el identificador. A nuestra imagen la vamos a llamar imageViewCamaraFotos.
- Vamos a añadir un TextView para añadir el texto que aparecía en el ejemplo. Para que nuestro nuevo elemento tenga como referencia la imagen que hemos incluido la primera, deberemos marcar uno de los puntos que tiene la imagen con uno de los puntos del TextView. De esa manera quedarán referenciados. Lo alinearemos con la parte izquierda y con la parte superior de la foto para que siempre estén a la misma altura.
- Cambiamos el id del TextView y le llamamos textViewTitulo.
- Añadiremos otro TextView que referenciaremos con el textViewTitulo en la parte superior y con la imagen a la izquierda.
- El resultado sería el siguiente

```
<ImageView
    android:id="@+id/imageViewCamaraFotos"
    android:layout_width="61dp"
    android:layout_height="75dp"
    android:layout_alignParentStart="true"
    android:layout_alignParentTop="true"
    android:layout_marginStart="23dp"
    android:layout_marginTop="28dp"
    app:srcCompat="@drawable/cameraswitch" />

<TextView
    android:id="@+id/textViewTitulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignTop="@+id/imageViewCamaraFotos"
    android:layout_alignEnd="@+id/imageViewCamaraFotos"
    android:layout_marginTop="1dp"
    android:layout_marginEnd="-101dp"
    android:text="TextView" />

<TextView
    android:id="@+id/textViewMarca"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/textViewTitulo"
    android:layout_marginStart="43dp"
    android:layout_marginTop="7dp"
    android:layout_toEndOf="@+id/imageViewCamaraFotos"
    android:text="TextView" />
```



ConstraintLayout I

- Permite crear diseños grandes y complejos con una jerarquía de vistas planas, es decir sin grupos de vistas anidadas.
- Es similar al RelativeLayout porque hay relaciones entre las vistas del mismo nivel y de niveles superiores, pero es más flexible y más fácil de usar.
- Se programa al 100% gráficamente, lo que hace que sea más intuitivo.
- Vamos a crear un proyecto nuevo que vamos a llamar ConstraintLayout.
- Una vez creado, comprobamos que en el árbol de componentes (component tree) del diseño gráfico del activity_main.xml aparece ConstraintLayout
- Por lo tanto Android Studio cuando generamos un nuevo proyecto, automáticamente nos crea un ConstraintLayout.

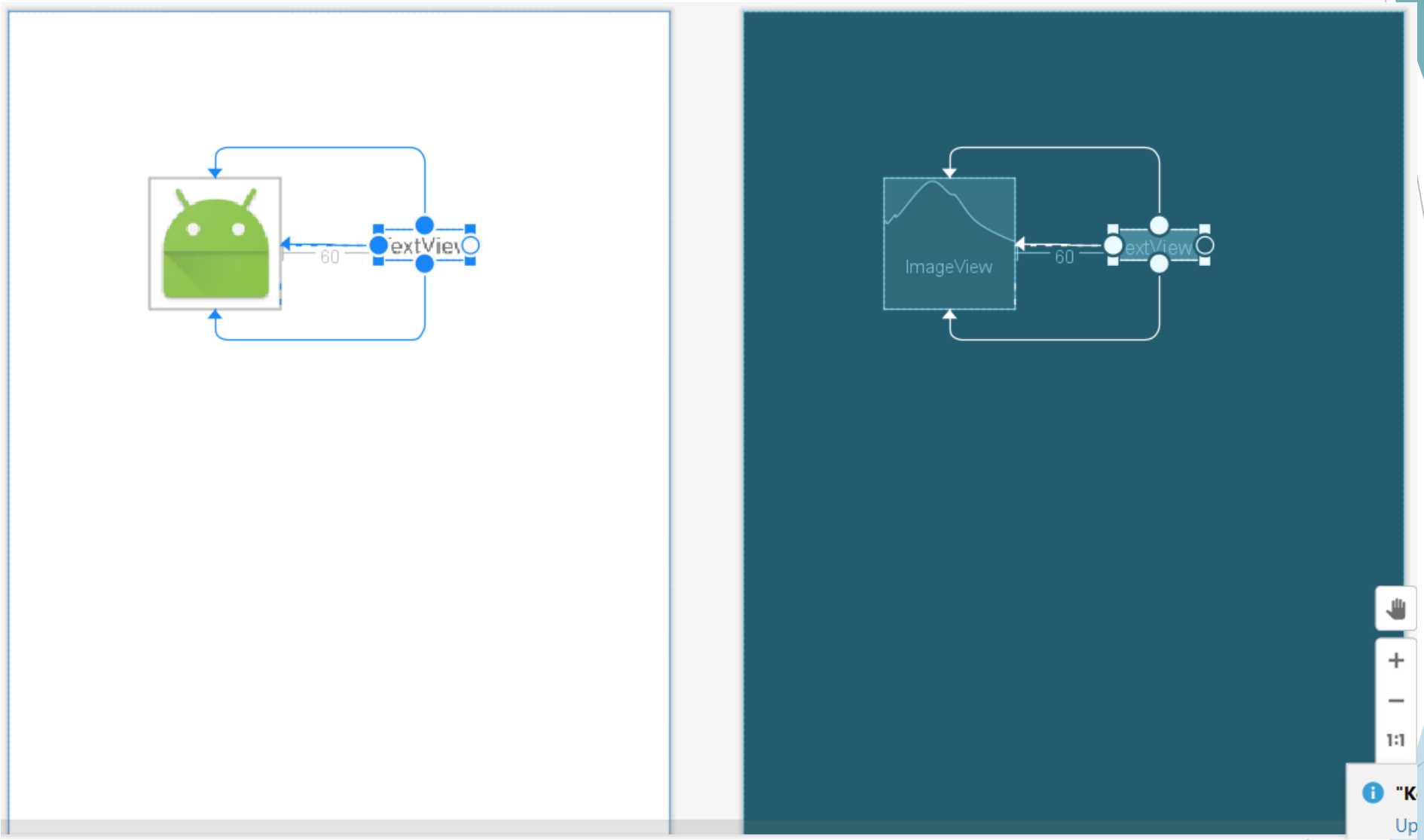


ConstraintLayout II

- Este tipo de layout es el recomendado para realizar el diseño de interfaces ya que su creación es más simple y facilita mucho el diseño.
- Desde el propio árbol de componentes podemos eliminar el TextView que tenemos creado siempre por defecto.
- Para nuestro ejemplo vamos a añadir una imagen.
- En este caso como es el primer elemento la relación la haremos con el elemento padre. Este paso es exactamente igual que en RelativeLayout.
- Nos aparecen las coordenadas de la imagen y a través de ellas la relacionaremos con el padre.
- Podemos poner las cuatro referencias y si queremos podemos eliminar alguna pulsando botón derecho sobre la referencia que deseemos eliminar.
- Generalmente en este primer elemento se deja solo los márgenes superior e izquierdo.

ConstraintLayout III

- Cómo crear una IU responsiva con ConstraintLayout (android.com)
- En este enlace nos explica como se generan todas las relaciones de los elementos de ConstraintLayout.



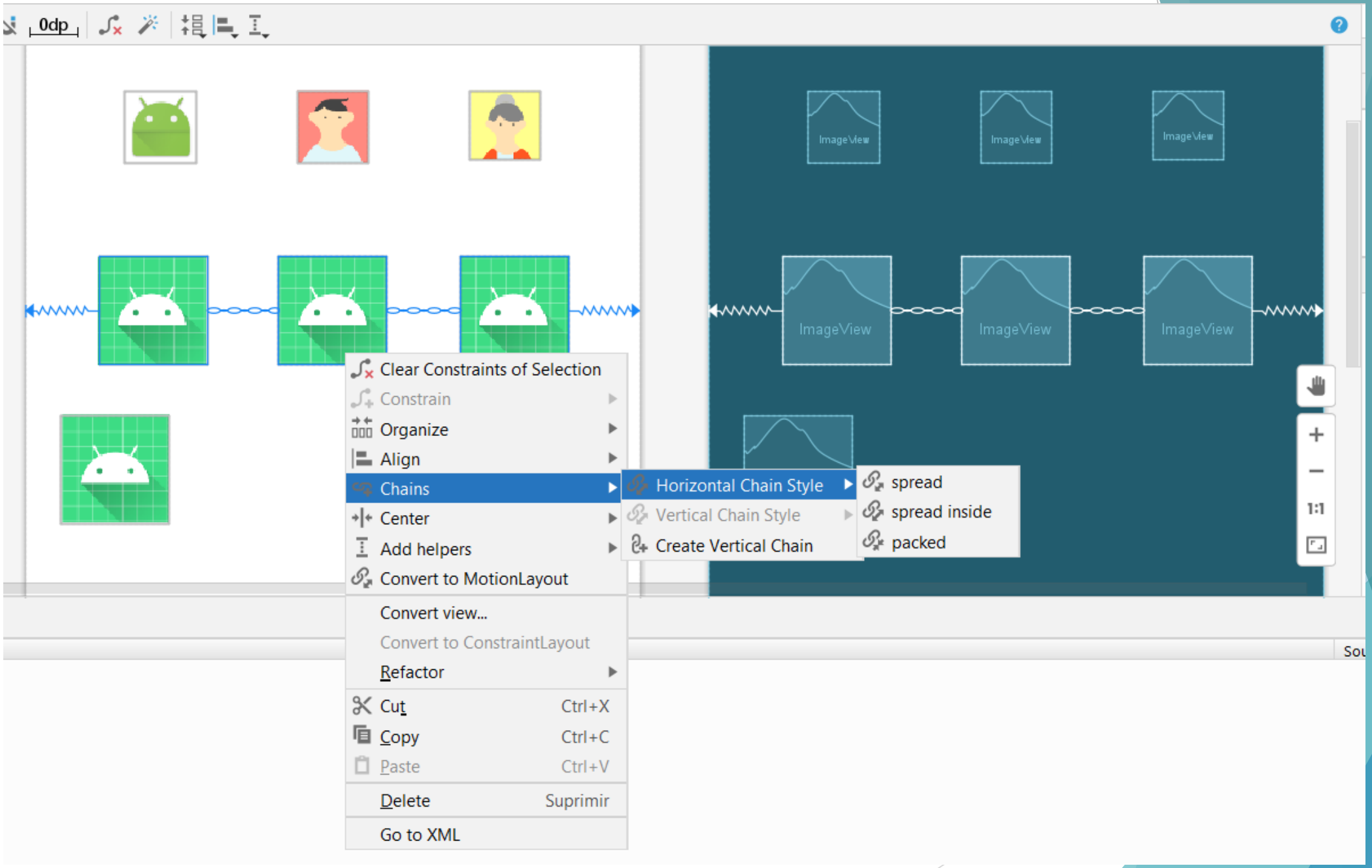
Estando los elementos de esta manera relacionados los diferentes elementos la imagen y el texto estarán alineados y el texto exactamente en el centro de la imagen.

Ejemplo Constraint Layout

- VAMOS A HACER UN EJEMPLO
- INSERTAREMOS TRES IMÁGENES DEL PROPIO ANDROID STUDIO.
- UNA VEZ INSERTADAS VAMOS A UNIRLAS UNAS A OTRAS A TRAVÉS DEL LAS OPCIONES QUE NOS FACILITA EL CONSTRAINT LAYOUT
- UNA VEZ UNIDAS VAMOS A CENTRARLAS HORIZONTALMENTE LAS TRES COMO SI FUERAN UN GRUPO.
- PROBAR DIFERENTES OPCIONES DE CENTRADO, POR EJEMPLO CON MÁRGENES O SIN MÁRGENES.
- ¿PARA QUE CREES QUE ES ÚTIL ESTA OPCIÓN?

Solución al ejemplo I

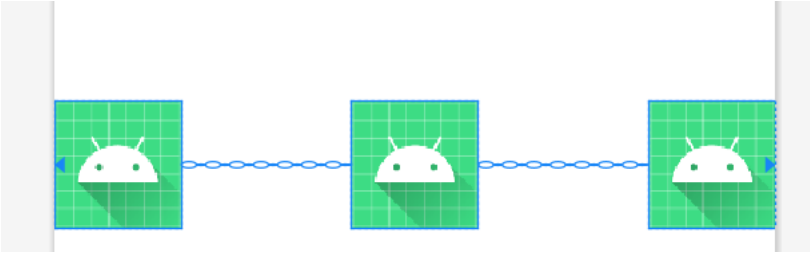
- Insertamos tres imágenes.
- La que está más a la izquierda la anclamos al margen izquierdo.
- La que está en el centro la anclamos a la izquierda con la imagen de la izquierda.
- La que está más a la derecha la anclamos a la izquierda con la imagen del centro y a la derecha con el margen derecho.
- Seleccionamos con el ratón las tres imágenes y haciendo click con el botón derecho del ratón, seleccionamos las opciones que aparecen en la imagen


0dp
ImageView
ImageView
ImageView
ImageView
ImageView
ImageView
Clear Constraints of Selection
Constrain
Organize
Align
Chains
Center
Add helpers
Convert to MotionLayout
Convert view...
Convert to ConstraintLayout
Refactor
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Delete Suprimir
Go to XML
Horizontal Chain Style
Vertical Chain Style
Create Vertical Chain
spread
spread inside
packed
1:1

Solución al ejemplo II

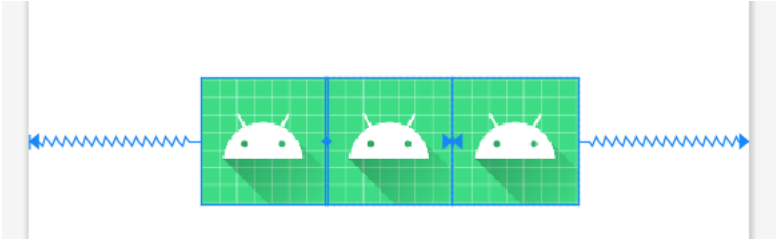
- Si seleccionamos la opción chains (cadenas), la opción horizontal chain styley dentro de esta la opción spread, lo que hará será centrar las tres imágenes teniendo la misma distancia entre ellas y los márgenes.

- Si seleccionamos la opción spread inside, desaparecerán los márgenes y dispondrá las tres imágenes a la misma distancia entre ellas.



Solución al ejemplo III

- Si seleccionamos la opción packed dispondrá las tres imágenes juntas en el centro de la pantalla.



- Lo que se consigue utilizando estas herramientas es que los iconos se ajusten dependiendo del tamaño del dispositivo móvil que vayamos a utilizar.
- Prueba a hacerlo ahora de manera vertical.

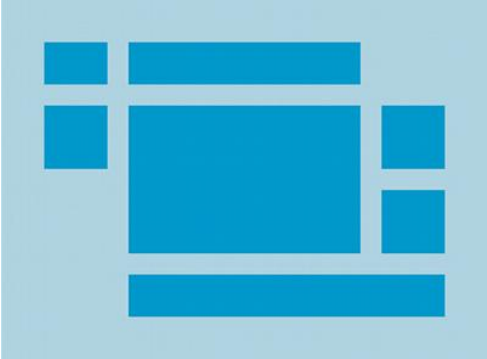
TableLayout I

- Posiciona sus elementos secundarios en filas y columnas.
- Los contenedores de TableLayout no muestran líneas de bordes para las filas, columnas o celdas.
- La tabla tendrá la misma cantidad de columnas que la fila con la mayor cantidad de celdas:
  - Se pueden dejar celdas vacías.
  - Las celdas pueden abarcar varias columnas, igual que en HTML, usando el campo span de la clase TableRow.LayoutParams.

TableLayout I

- Los objetos TableRow son las vistas secundarias de TableLayout (cada objeto TableRow define una sola fila de la tabla).

- Cada fila tiene cero o más celdas, y cada celda se define por cualquier tipo de objeto View (como ImageView o TextView).



```
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:stretchColumns="1">
    <TableRow>
        <TextView
            android:text="@string/table_layout_4_open"
            android:padding="3dip" />
        <TextView
            android:text="@string/table_layout_4_open_shortcut"
            android:gravity="right"
            android:padding="3dip" />
    </TableRow>

    <TableRow>
        <TextView
            android:text="@string/table_layout_4_save"
            android:padding="3dip" />
        <TextView
            android:text="@string/table_layout_4_save_shortcut"
            android:gravity="right"
            android:padding="3dip" />
    </TableRow>
</TableLayout>
```

TableLayout I

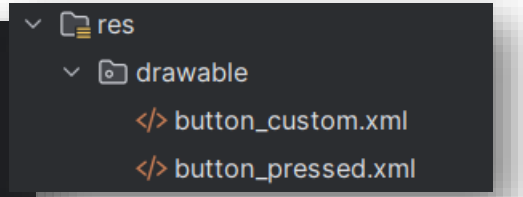
- La tabla tendrá la misma cantidad de columnas que la fila con la mayor cantidad de celdas:
  - Se pueden dejar celdas vacías.
  - Las celdas pueden abarcar varias columnas, igual que en HTML, usando el campo span de la clase TableRow.LayoutParams.

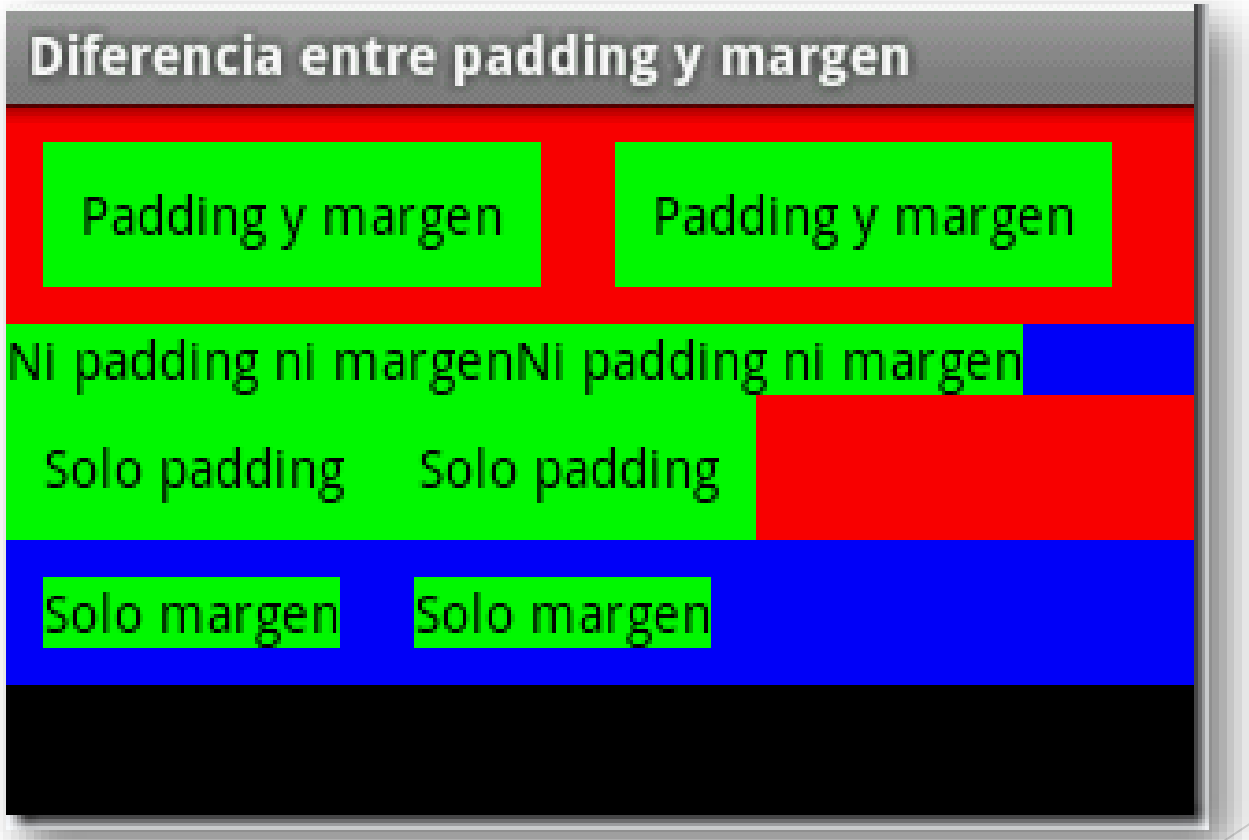
```
<TableRow android:paddingTop="20dp"
android:gravity="center_horizontal">

    <EditText
         android:layout_width="150dp"
         android:layout_height="40dp"
         android:layout_span="2">
    </EditText>

</TableRow>
```

Componentes customizados

- Es posible modificar las propiedades de los componentes mediante drawables para adaptarlos a nuestras necesidades y reutilizarlos.

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:background="@drawable/button_custom"
```




```
<selector xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:drawable="@drawable/button_pressed"
        android:state_pressed="true" />

    <item>
        <shape android:shape="rectangle">
            <size android:height="25dp" android:width="50dp" />
            <solid android:color="@color/button2" />
            <stroke android:width="2dp" android:color="@color/button1" />
            <corners android:radius="8dp" />

        </shape>
    </item>

</selector>
```

Propiedades: padding y margen